# Dataset Description

This competition evaluates AI software engineering agents on their ability to solve real-world Python bug fixes and feature requests.

To assist agents in large codebase navigation, dependency understanding, and semantic retrieval, the competition provides **code graphs**, **graph/node embeddings**, and specialized **graph tools** alongside traditional repository exploration mechanisms.

Participants are provided with a **training set** (including tasks, code graphs, and embeddings) for local agent development, prompt engineering, and offline validation. Submissions are evaluated against a **test set** curated under a similar pipeline, with identical graph generation methods, and verification standards.

In this public-facing version of the dataset, you will only see the training set. When your submission is scored, this training set will be replaced with the hidden test set.

## Files

**tasks.jsonl**

Line-delimited JSON file containing the 129 public development tasks. Each JSON object represents one bug-fixing benchmark instance and includes both the reference fix (`patch`) and verification unit tests (`test_patch`) for local eval\ uation.

- `instance_id` - unique identifier for the benchmark task combining the repository short name and issue or pull request number (for example, `fastapi_11194` or `rich_3454`).
- `repo` - source GitHub repository in `owner/repo` format (`fastapi/fastapi`, `Textualize/rich`, `psf/requests`, or `encode/httpx`).
- `base_commit` - 40-character hexadecimal Git commit SHA representing the repository snapshot immediately prior to the bug fix.
- `problem_statement` - natural-language bug report or feature issue description provided to the agent as the task prompt.
- `hints_text` - optional issue comments or maintainer hints providing additional context (may be empty).
- `patch` - reference unified `git diff` solution patch that resolves the issue on `base_commit` (provided in `published/tasks.jsonl` for local training and analysis; excluded from the hidden `rerun/tasks.jsonl` test set).

- `test_patch` - unified `git diff` patch containing the verification unit tests executed by `pytest` during Phase 2 grading (provided in `published/tasks.jsonl` for local verification; kept in the private `secret/solution.parquet` archive during competition scoring).
- `created_at` - ISO-8601 timestamp indicating when the issue or pull request was created.

**snapshots/** Directory containing 129 compressed Git working tree archives (`snapshots/<instance_id>.tgz`, such as `fastapi_11194.tgz`, `requests_7205.tgz`, `rich_2725.tgz`, and `httpx_3672.tgz`). Each archive contains a full Git repository frozen at `base_commit` with forward commit history removed so agents can safely run `git status`, `git log`, and `git diff` inside `/workspace` without leaking future fixes.

**graphs/** Directory containing serialized NetworkX Abstract Syntax Tree (AST) call and dependency graphs in JSON format (256 files total: 129 task-named files `<instance_id>.json` hard-linked to 127 commit-named files `<repo_short>_<base_commit>.json`). These graphs back the `get_code_neighbors` and `get_code_subgraph` agent tools and use the following schema:

- `directed` - boolean flag indicating whether graph edges are directed (`true`).
- `multigraph` - boolean flag indicating whether multiple edges can connect the same pair of symbols (`true`).
- `graph` - dictionary of global graph metadata attributes.
- `nodes` - list of code entity node objects in the repository snapshot.
- `nodes[*].id` - unique fully qualified Python symbol path for the module, class, or function (for example, `fastapi.routing._prepare_response_content`).
- `nodes[*].name` - human-readable qualified symbol name matching `id`.
- `nodes[*].text` - full Python source code definition of the function, method, or class at `base_commit`.
- `edges` - list of directed structural relationship objects between code nodes.
- `edges[*].source` - origin symbol `id` where the call or reference originates.
- `edges[*].target` - destination symbol `id` being called, imported, or referenced.
- `edges[*].type` - structural relationship category linking `source` to `target` (such as `calls`).
- `edges[*].key` - integer index ($0, 1, \dots$) distinguishing parallel edges in the multigraph.

**embeddings/** Directory containing compressed NumPy archives (256 files total: 129 task-named files `<instance_id>.npz` hard-linked to 127 commit-named files

`<repo_short>_<base_commit>.npz`). These archives power offline semantic code retrieval via the `search_similar_code` tool.

- `<node_id>.npy` - `float32` NumPy array of length 256 containing dense semantic vector features `embedding_[0-255]` for the corresponding AST graph symbol `node_id`.

**wheels/** Directory of 124 pre-compiled offline Python binary wheels (`*.whl`) covering historical dependency versions for `fastapi`, `starlette`, `pydantic`, `requests`, `urllib3`, `rich`, `httpx`, `httpcore`, `pytest`, and transitive packages. Mounted read-only at `/wheels/` inside the sandbox container so `pip` can install repository dependencies without internet access.

**docker/** Container build specifications and Python standard library compatibility shims used to build the local `swebench-sandbox:latest` image:

- `Dockerfile.sandbox` - base sandbox container specification providing Python 3.13, `git`, `pytest`, and build backends (`setuptools`, `hatchling`, `flit-core`, `poetry-core`, `pdm-backend`).
- `Dockerfile.public` - public development container variant pre-configured for the 129 open-source tasks.
- `imp.py` - compatibility shim restoring the legacy `imp` module removed in Python 3.13 for older test suites.
- `telnetlib.py` - compatibility shim restoring the legacy `telnetlib` module removed in Python 3.13.

**sandbox/setup.py** Container initialization script executed inside `/workspace` when preparing a task sandbox. Inspects `pyproject.toml`, `setup.cfg`, or `requirements.txt`, resolves compatible offline wheels from `/wheels/`, performs an editable install (`pip install --no-index --find-links=/wheels -e .`), and creates a clean baseline Git commit so `git diff HEAD` captures only your agent's edits.

**sample_submission/** Ready-to-run baseline ADK submission directory illustrating the declarative YAML schema, multi-agent delegation, and multi-LoRA routing.

**HARNESS_README.md** In-depth technical guide covering the `swegemma`, `adk-submission`, and `adk-eval-core` libraries, the two-container sandbox lifecycle, tool call signatures, context compaction, and local CLI evaluation commands.

**submission.parquet** Output file automatically produced at `/kaggle/working/submission.parquet` during evaluation when the inference script runs your uploaded `submission.zip` archive against the benchmark tasks:

- `id` - unique task identifier matching `instance_id` in `tasks.jsonl`.
- `prediction` - unified `git diff` patch string generated by your agent relative to `base_commit` (or `NO_PATCH` if no changes were produced).

## Dataset Generation & Verification Pipeline

Both the training set and test set were produced using an automated curation and two-phase verification harness:

1. **Commit & PR Mining**: Filtered repository histories for commits that jointly modify core functional logic (`.py`) and corresponding unit tests (`test_*.py` / `*_test.py`).
2. **Issue Linking & Prompt Extraction**: Matched each code change with its original problem description, issue tracker thread, or bug report.
3. **Filtering & De-noising**: Excluded mechanical refactors, documentation-only changes, large-scale automated changes (LSCs), and ambiguous issues lacking reproducible acceptance criteria.
4. **Execution Verification Harness**:
    - **Fail-to-Pass (Baseline Verification)**: The test reproducer (`test_patch`) is applied to `base_commit` without the fix; tests **must fail** (non-zero exit code).
    - **Pass-to-Pass (Solution Verification)**: The reference fix (`patch`) is applied with `test_patch`; the entire test suite **must pass cleanly** (exit code 0).
    - **Near Completion with Frontier Models**: For the test set, we check that a larger frontier model can pass the case or get within a single test case of passing.

There are about 120 tasks in the test set, evenly divided between the public and private splits. The test set was curated from a set of private repositories.